Agentes Conversacionais

UNIDADE 03

Aplicando técnicas de processamento de linguagem natural e machine learning na construção de agentes conversacionais

*Os agentes conversacionais podem ser classificados em **agentes de diálogo** ou **chatbots**, além disso, podem ter diferentes **modelos conversacionais** e **abordagens de desenvolvimento**, conforme a Unidade de Estudo introdução aos chatbots e agentes de diálogo.*
*Nesta Unidade, vamos focar nas técnicas de **Processamento de Linguagem Natural (PLN)** e **Machine Learning (ML)** aplicadas no desenvolvimento de agentes conversacionais, independentemente de suas classificações, e com isso construir alguns exemplos de agentes utilizando a linguagem de programação **Python**. Toda arquitetura e processos serão bem descritos e documentados, e para cada exemplo será fornecido um Jupyter Notebook, que pode ser executado diretamente no ambiente do **Google Colab**.*
Então, vamos lá?
*Será apresentada, primeiramente, uma visão geral das técnicas de PLN e ML comumente utilizadas, e em seguida, algumas **arquiteturas** e processos e recursos comuns no **pipeline** de um agente, como o **reconhecimento de intenções** e **dicionários de respostas**. Enfim, vamos colocar a mão na massa em alguns códigos em Python para executarmos alguns exemplos de agentes conversacionais.*

Técnicas de processamento de linguagem natural e *machine learning*

A utilização de técnicas de PLN se torna evidente, pois um agente precisa processar entradas de textos feitas pelo usuário. Já as de ML proporcionam mais robustez a modelos de agentes, pois conseguem lidar com entradas inesperadas e aprender "sozinhas". A seguir, estão elencadas algumas das técnicas mais utilizadas no desenvolvimento de agentes.

Pattern matching

O *pattern matching* é uma técnica muito simples, porém ainda muito utilizada no campo dos agentes e PLN em geral. Nela, utilizamos algoritmos de busca e **expressões regulares** para encontrar os padrões de escrita do usuário e achar um *template* correspondente para resposta. Usaremos esta técnica nos exemplos práticos desta Unidade.

"Compre duas passagens de classe econômica para o voo **GOL 1125.**"

`Expressão regular para obter informações de voos → **voo [A-Z]{3} [0-9]{4}.**`

Pré-processamento

O pré-processamento corresponde a todo e qualquer processo de "limpeza" e normalização do texto, antes de passá-lo às próximas etapas do *pipeline* de execução. A seguir, um exemplo que mostra um texto informado pelo usuário, e na sequência, esse mesmo texto após a retirada de caracteres especiais e interjeições indesejadas, além da normalização para letras minúsculas.

`Texto original → *EI MANO TU NÃO SABE O QUE ACONTECEU =O*.`

`Texto pré-processado → *tu não sabe o que aconteceu*.`

Part-Of-Speech Tagging (POS-Tagging)

O *Part-Of-Speech Tagging* (*POS-Tagging*) é o algoritmo responsável por **inferir as classificações morfológicas das palavras em um texto**, ou seja, ele atribui a cada palavra um "*part of speech*". Por exemplo: o *POS-Tagging* classifica se a palavra é um verbo, substantivo, adjetivo e assim por diante. O exemplo a seguir mostra um texto com as *tags* correspondentes, colocadas após as palavras, separadas por uma barra invertida.

`Olá\*UH* meu\*PRP* nome\*NN* é\*VBZ* Lucas\*NNP*.\*SENT*`

Cada palavra é rotulada com a *tag* correspondente, neste caso, utilizando o *tagset* do Penn Treebank. Você pode acessar o *link* e verificar que UH significa interjeição, NN, nome próprio, PRP, pronome pessoal etc. Há vários modelos de *POS-Tagging* disponíveis para uso, em diversos idiomas, e com taxas de acerto bem altas. Existe ainda a possibilidade de estender o conhecimento sobre a palavras ao executar um ***parser* sintático**, que encontra a estrutura sintática das sentenças, para podermos identificar situações como a dependência entre as palavras em um texto.

Esta técnica pode auxiliar o seu agente a **encontrar informações necessárias para processar uma resposta ao usuário**, como resgatar o nome do usuário, ao encontrar as palavras associadas a *tag* de nome próprio.

Named Entity Recognition (NER)

A tarefa de *Named Entity Recognition* (NER), ou reconhecimento de entidades nomeadas, em português, é muito similar ao *POS-Tagging*, que também é considerado um algoritmo de *sequence labeling*, pois rotula palavras em um texto segundo alguma classificação. A diferença é que **o NER rotula apenas as palavras, ou sequência de palavras, que correspondam a uma série de entidades pré-definidas**, ou seja, nomes de "coisas", como nomes de pessoas, empresas, cidades.

[Lucas]\NOME reside em [São José dos Pinhais]\CIDADE e trabalha na [PUCPR]\EMPRESA.

As entidades podem ser adaptadas para o contexto necessário, por exemplo: em um *chatbot* para a área médica, é interessante identificar conceitos clínicos importantes para processar a resposta ao usuário, conforme o exemplo a seguir.

Estou com [febre]\SINTOMA e [náusea]\SINTOMA. Estou tomando [paracetamol]\MEDICAMENTO para [dor de cabeça]\SINTOMA.

As abordagens de NER incluem a criação de grandes **listas de palavras** com sua classificação associada, utilização de **expressões regulares** para encontrar padrões de escrita, ou a utilização de um modelo treinado de **ML** para fazê-lo.

Bag of words e TF-IDF

O *Bag Of Words* (BOW) e o TF-IDF são duas técnicas de representação vetorial de textos que podem ser utilizadas de diversas maneiras, mas principalmente como forma de **encontrar atributos e características importantes no texto**, e utilizar esses atributos como **entrada para o treinamento de algoritmos de ML**, conforme exemplos que veremos mais para frente. Além disso, podem auxiliar no cálculo de similaridade entre palavras.

O **BoW** é a mais simples das técnicas de transformação vetorial de textos, sendo necessários apenas dois passos para realização da transformação:

1. Determinar o vocabulário do(s) texto(s).

2. Realizar a contagem dos termos (frequência das palavras).

Ou seja, um *corpus* com os seguintes documentos:

- O menino correu.
- O menino correu do cão.
- O menino com o cão.

Teria a seguinte representação vetorial BoW:

Tabela 1 – Exemplo de representação vetorial BoW de um *corpus*

#ParaTodosVerem

Documento	o	menino	correu	do	cão	com
O menino correu	1	1	1	0	0	0
O menino correu do cão	1	1	1	1	1	0
O menino com o cão	2	1	0	0	1	1

Fonte: Autor (2021).

Já o **TF-IDF**, por sua vez, é similar ao BoW, com algumas melhorias. Ele consegue avaliar a importância de uma palavra em um documento dentro de um *corpus*. A importância aumenta proporcionalmente ao número de vezes que uma palavra aparece no documento, mas é compensada pela frequência da palavra no *corpus*.

Por exemplo: queremos calcular a importância da palavra "carro" em um documento contido em um *corpus* de um milhão de documentos. A palavra aparece cinco vezes em um documento com 100 palavras, portanto, seu TF pode ser calculado como:

```
TF(carro) = 5 / 100 = 0.05
```

Verificou-se que a palavra "carro" aparece em 500 documentos do *corpus*, portanto, para calcular seu IDF, fazemos:

```
IDF(carro) log_e(1000000 / 500) = 3.3
```

E, enfim, o TF-IDF desta palavra é o produto destes valores:

```
TF-IDF(carro) = 0.05 * 3.3 = 0.16
```

Redes neurais artificiais e Large Language Models

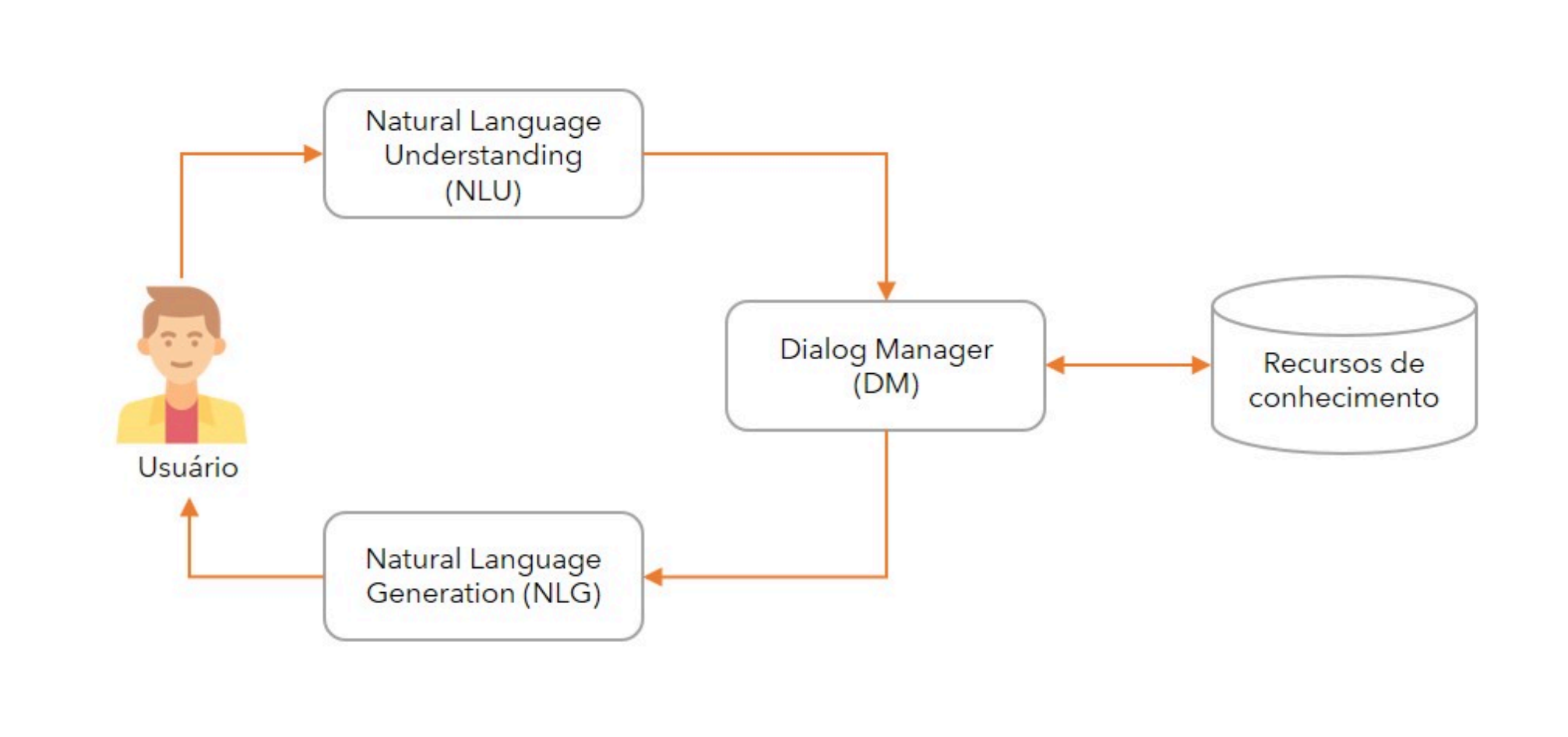
A arquitetura de **redes neurais artificias** causou um grande impacto na área de PLN (e outras da IA), pois tem a habilidade de aprender a executar tarefas por meio do treinamento baseado em dados, com resultados superiores a outras arquiteturas de ML. Em Unidades de Aprendizagem futuras, trabalharemos com exemplos que fazem uso desta tecnologia, incluindo os famosos Large Language Models (LLMs), como o GPT.

Arquiteturas e *pipeline* de execução

Existem diferentes arquiteturas e *pipelines* de execução em agentes conversacionais, a seguir, na Figura 1, você verá um exemplo tradicional de arquitetura geral.

Figura 1 – Arquitetura de um agente conversacional

#ParaTodosVerem



Esquema de arquitetura tradicional de um agente conversacional, em que NLU pode ser traduzido como “entendimento de linguagem natural”, DM, como “gerenciador de diálogo” e NLG, como “geração de linguagem natural”. Fonte: Adaptado de Mctear (2020).

Cada módulo dos apresentados no esquema apresentado (*i.e.*, NLU, DM, MLG) são compostos por um *pipeline* de execução, com diversas etapas de processamento de texto, inferência de dados e geração de uma saída para usuário.

Natural Language Understanding (NLU)

No caso do módulo de NLU, podemos citar os seguintes processos, já abordados na seção anterior:

- Pré-processamento/limpeza de dados.
- Segmentação de sentenças/palavras.
- *POS-Tagging*.
- Análise sintática.
- NER.

No final das contas, o módulo de **NLU** tem por objetivo extrair a **propósito do texto**, ou seja, saber o que o usuário deseja. Portanto, dois conceitos importantes desta etapa são as **intenções** (***intents***) e as **entidades** (***entities***), que podem ser extraídos de diversas maneiras. Vamos, então, identificar as intenções e entidades do exemplo a seguir.

`Gostaria de reservar um voo para São Paulo, amanhã, às oito horas.`

Intenção (***intent***) -> Neste caso, o objetivo do usuário é comprar uma passagem aérea para São Paulo no dia e horário estabelecidos. Assim, a intenção seria "comprar passagem aérea". Ou seja, o agente deve mapear a mensagem do usuário a uma intenção pré-estabelecida, para assim executar a tarefa pedida.

Entidade (***entity***) -> Neste exemplo, encontramos três entidades importantes para execução da tarefa, a cidade de destino, data e horário.

Portanto, a intenção "**comprar passagem aérea**" possui as entidades **cidade de destino**, **data** e **horário**. Além destas, poderíamos ter ainda a **cidade de origem**, que poderia ser inferida automaticamente pela localização atual do usuário.

Dialog Manager (DM)

O DM, ou **gerenciador de diálogo**, é o componente que recebe as interpretações feitas pelo NLU e interage com bases de conhecimento externas para realização das tarefas e toma as decisões quanto à resposta ao usuário.

Ainda utilizando o exemplo anterior, o DM poderia antes de agendar um voo para o usuário, confirmar se a saída do voo é da sua localização atual, visto que ele não informou explicitamente a sua cidade de origem no texto.

Este módulo ainda pode manter um registro de interações anteriores com o usuário para utilizá-las como base para novas decisões. Bancos de dados de domínio também podem ser armazenados pelo DM, como por exemplo uma base de dados de voos disponíveis.

Natural Language Generation (NLG)

O papel do último modulo é receber uma resposta do módulo de DM, e converter esta resposta em palavras (*i.e.*, linguagem natural). Este módulo pode seguir abordagens mais simples com o uso de *templates*, por exemplo, em que apenas encaixamos informações em *slots* (*i.e.*, "o voo que você quer agendar tem saída da cidade de [$CIDADE]"), ou ainda algoritmos mais sofisticados de geração ou síntese de textos.

Na prática

Nesta seção você irá executar o código-fonte em Python de alguns exemplos de agentes conversacionais, e verificar suas diferentes arquiteturas, *pipelines* e técnicas utilizadas. Acesse os *notebooks* fornecidos, execute-os e faça alterações no código para verificar o resultado final.

Neste vídeo, serão trabalhados dois exemplos de implementação de agentes conversacionais, similares aos apresentados no vídeo anterior, em que poderemos nos aprofundar um pouco mais na utilização das técnicas de PLN.

Construindo agentes conversacionais na prática
PUCPR Online

Assista no

EXEMPLO

Chatbot de atendimento ao cliente baseados em regra

Aqui, trabalharemos com um ***chatbot* de atendimento a clientes** que acessam o *website* de um restaurante. Utilizaremos uma abordagem **baseada em regras e mapeamento de intenções e** respostas para cobrir dúvidas simples que o cliente possa vir a ter.

Acesse o *notebook* que contém o código-fonte do exemplo 1 de *chatbot.*

Google Colaboratory

Fonte: Autor (2021).

EXEMPLO

Agente de diálogo híbrido (baseado em regras e dados)

Neste exemplo, trabalhamos com um agente de diálogo (um **agente conversacional baseado em tarefas**), que serve como um consultor de determinado assunto. Nele, utilizamos uma abordagem **baseada em regras como arquitetura principal**, mas para uma das regras, **exploramos dados em um *corpus*** para dar as repostas ao usuário.

Você vai perceber que ao utilizar uma base de dados, nosso agente ganha muita flexibilidade e robustez.

Acesse o *notebook* que contém o código-fonte do exemplo 2 de agente de diálogo.

Google Colaboratory

Fonte: Autor (2021).

EXPERIMENTE

Em ambos os exemplos apresentados anteriormente, há uma seção chamada “**o que pode ser feito agora?**” no *notebook*.

Escolha um ponto de melhoria de cada exemplo e implemente as sugestões.

Termina aqui a Unidade de Aprendizagem, na qual você:

- Verificou as técnicas de PLN e ML mais utilizadas na construção de agentes conversacionais.

- Entendeu quais as principais funções de cada módulo em uma arquitetura de agente.
- Desenvolveu exemplos de *chatbots* e agentes de diálogo baseados em regras e em dados.

Referências

JURAFSKY, D; MARTIN J H. **Speech and Language Processing**. 3. ed. California: Stanford, 2020. Disponível em: https://web.stanford.edu/~jurafsky/slp3/. Acesso em: 15 dez. 2020.

MCTEAR, M**. Conversational AI**: dialogue systems, conversational agents, and chatbots. California: Morgan & Claypool, 2020.

PEREZ-MARIN, D; PASCUAL-NIETO, I. **Conversational agents and natural language interaction**: techniques and effective practices. Pennsylvania: Information Science Reference, 2011.

© PUCPR - Todos os direitos reservados.